

STRUCTURES DE DONNÉES

Jérémy Cabessa
Laboratoire DAVID, UVSQ

STRUCTURES DE DONNÉES

- ▶ Une **structure de données (data structure)** permet de stocker en machine des *ensembles dynamiques de données*.
- ▶ Ensemble *dynamique* de données : qui peut évoluer au cours du temps. Insérer, modifier, supprimer des éléments...
- ▶ Exemples (Python) : listes, dictionnaires, sets.

STRUCTURES DE DONNÉES

- ▶ Une **structure de données (data structure)** permet de stocker en machine des *ensembles dynamiques de données*.
- ▶ Ensemble *dynamique* de données : qui peut évoluer au cours du temps. Insérer, modifier, supprimer des éléments...
- ▶ Exemples (Python) : listes, dictionnaires, sets.

STRUCTURES DE DONNÉES

- ▶ Une **structure de données (data structure)** permet de stocker en machine des *ensembles dynamiques de données*.
- ▶ Ensemble *dynamique* de données : qui peut évoluer au cours du temps. Insérer, modifier, supprimer des éléments...
- ▶ **Exemples (Python)** : listes, dictionnaires, sets.

TABLEAUX

- Un **tableau (array)** est une structure de données.

indices	0	1	2	3	4	5	6	7
valeurs	12	-7	33	11	-42	1	0	8

- Représente un *ensemble dynamique, ordonné et de taille fixe* d'éléments de même type.
- Usage : rapidité de traitement en ce qui concerne l'accès aux éléments.

TABLEAUX

- Un **tableau (array)** est une structure de données.

indices	0	1	2	3	4	5	6	7
valeurs	12	-7	33	11	-42	1	0	8

- Représente un *ensemble dynamique, ordonné et de taille fixe* d'éléments de même type.
- Usage : rapidité de traitement en ce qui concerne l'accès aux éléments.

TABLEAUX

- Un **tableau (array)** est une structure de données.

indices	0	1	2	3	4	5	6	7
valeurs	12	-7	33	11	-42	1	0	8

- Représente un *ensemble dynamique, ordonné et de taille fixe* d'éléments de même type.
- Usage : rapidité de traitement en ce qui concerne l'accès aux éléments.

TABLEAUX

- ▶ Taille fixe.
- ▶ Accès aux éléments de manière *directe*, par adressage des cellules via des *indices* : simple.
`t[3]`, `t[3:7]`, ...
- ▶ Insertion et suppression en *milieu* de tableau nécessitent de décaler tous les éléments suivants : coûteux.
`t.insert(0,14)`, `t.pop(0)`, ...
- ▶ Remarque (Python) : une liste Python est un tableau *dynamique* : lorsqu'il est plein, un tableau plus grand est alloué ("realloc") et les éléments sont recopiés. Ce coût est réparti sur l'ensemble des ajouts, donc `t.append()` et `t.pop()` sont en $O(1)$ *amorti*.

TABLEAUX

- ▶ Taille fixe.
- ▶ Accès aux éléments de manière *directe*, par adressage des cellules via des *indices* : simple.

`t[3]`, `t[3:7]`, ...

- ▶ Insertion et suppression en *milieu* de tableau nécessitent de décaler tous les éléments suivants : coûteux.

`t.insert(0,14)`, `t.pop(0)`, ...

- ▶ Remarque (Python) : une liste Python est un tableau *dynamique* : lorsqu'il est plein, un tableau plus grand est alloué ("realloc") et les éléments sont copiés. Ce coût est réparti sur l'ensemble des ajouts, donc `t.append()` et `t.pop()` sont en $O(1)$ *amorti*.

TABLEAUX

- ▶ Taille fixe.
- ▶ Accès aux éléments de manière *directe*, par adressage des cellules via des *indices* : simple.

`t[3]`, `t[3:7]`, ...

- ▶ Insertion et suppression en *milieu* de tableau nécessitent de décaler tous les éléments suivants : coûteux.

`t.insert(0,14)`, `t.pop(0)`, ...

- ▶ Remarque (Python) : une liste Python est un tableau *dynamique* : lorsqu'il est plein, un tableau plus grand est alloué ("realloc") et les éléments sont copiés. Ce coût est réparti sur l'ensemble des ajouts, donc `t.append()` et `t.pop()` sont en $O(1)$ *amorti*.

TABLEAUX

- ▶ Taille fixe.
- ▶ Accès aux éléments de manière *directe*, par adressage des cellules via des *indices* : simple.
`t[3]`, `t[3:7]`, ...
- ▶ Insertion et suppression en *milieu* de tableau nécessitent de décaler tous les éléments suivants : coûteux.
`t.insert(0,14)`, `t.pop(0)`, ...
- ▶ **Remarque (Python)** : une liste Python est un tableau *dynamique* : lorsqu'il est plein, un tableau plus grand est alloué ("realloc") et les éléments sont recopiés. Ce coût est réparti sur l'ensemble des ajouts, donc `t.append()` et `t.pop()` sont en $O(1)$ *amorti*.

TABLEAUX

- ▶ **Principe** : les éléments sont stockés en mémoire de façon *contigüe* et *accessibles via un indice*.
- ▶ Les *éléments* du tableau sont des *objets* qui possèdent une *valeur* et un *indice* (et une adresse mémoire).
- ▶ Les valeurs sont accessibles via leurs indices.

élément x

x.valeur	x.indice
----------	----------

TABLEAUX

- ▶ **Principe** : les éléments sont stockés en mémoire de façon *contigüe* et *accessibles via un indice*.
- ▶ Les *éléments* du tableau sont des *objets* qui possèdent une *valeur* et un *indice* (et une adresse mémoire).
- ▶ Les valeurs sont accessibles via leurs indices.

élément x

x.valeur	x.indice
----------	----------

TABLEAUX

- ▶ **Principe** : les éléments sont stockés en mémoire de façon *contigüe* et accessibles via un *indice*.
- ▶ Les *éléments* du tableau sont des *objets* qui possèdent une *valeur* et un *indice* (et une adresse mémoire).
- ▶ Les valeurs sont accessibles via leurs indices.

élément x

x.valeur	x.indice
----------	----------

TABLEAUX : RECHERCHER, INSERER, SUPPRIMER EN $O(n)$

```
1 def rechercher(T,x):
2     for i in range(len(T)):
3         if T[i] == x:
4             return i
5     return None
```

```
1 def inserer(T, x, y):
2     """On suppose que T a la place pour inserer..."""
3     i = T.index(y)
4     for j in range(len(T)-1, i, -1):
5         T[j] = T[j-1]
6     T[i] = x
7     return T
```

```
1 def supprimer(T, x):
2     i = T.index(x)
3     for j in range(i, len(T)-1):
4         T[j] = T[j+1]
5     T[len(T)-1] = None
6     return T
```

TABLEAUX : RECHERCHER, INSERER, SUPPRIMER EN $O(n)$

```
1 def rechercher(T,x):
2     for i in range(len(T)):
3         if T[i] == x:
4             return i
5     return None
```

```
1 def inserer(T, x, y):
2     """On suppose que T a la place pour inserer..."""
3     i = T.index(y)
4     for j in range(len(T)-1, i, -1):
5         T[j] = T[j-1]
6     T[i] = x
7     return T
```

```
1 def supprimer(T, x):
2     i = T.index(x)
3     for j in range(i, len(T)-1):
4         T[j] = T[j+1]
5     T[len(T)-1] = None
6     return T
```

TABLEAUX : RECHERCHER, INSERER, SUPPRIMER EN $O(n)$

```
1 def rechercher(T,x):
2     for i in range(len(T)):
3         if T[i] == x:
4             return i
5     return None
```

```
1 def inserer(T, x, y):
2     """On suppose que T a la place pour inserer..."""
3     i = T.index(y)
4     for j in range(len(T)-1, i, -1):
5         T[j] = T[j-1]
6     T[i] = x
7     return T
```

```
1 def supprimer(T, x):
2     i = T.index(x)
3     for j in range(i, len(T)-1):
4         T[j] = T[j+1]
5     T[len(T)-1] = None
6     return T
```

REMARQUES

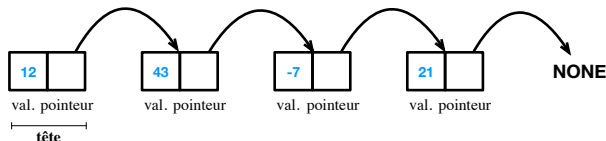
- ▶ L'accès aux éléments est simple : $O(1)$.
- ▶ Les procédures de recherche, insertion et suppression sont coûteuses : $O(n)$.

REMARQUES

- ▶ L'accès aux éléments est simple : $O(1)$.
- ▶ Les procédures de recherche, insertion et suppression sont coûteuses : $O(n)$.

LISTES CHAÎNÉES

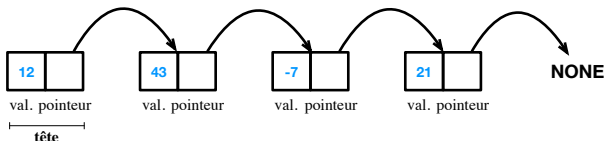
- ▶ Une **liste chaînée (linked list)** est une structure de données.



- ▶ Représente un *ensemble dynamique, ordonné et de taille arbitraire* d'éléments de même type.
- ▶ Usage : rapidité de traitement, lorsque l'ordre des éléments est important et que les insertions et les suppressions d'éléments sont plus fréquentes que les accès.

LISTES CHAÎNÉES

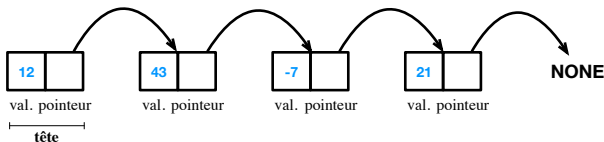
- ▶ Une liste chaînée (**linked list**) est une structure de données.



- ▶ Représente un *ensemble dynamique, ordonné et de taille arbitraire* d'éléments de même type.
- ▶ Usage : rapidité de traitement, lorsque l'ordre des éléments est important et que les insertions et les suppressions d'éléments sont plus fréquentes que les accès.

LISTES CHAÎNÉES

- ▶ Une liste chaînée (linked list) est une structure de données.



- ▶ Représente un *ensemble dynamique, ordonné et de taille arbitraire* d'éléments de même type.
- ▶ Usage : rapidité de traitement, lorsque l'ordre des éléments est important et que les insertions et les suppressions d'éléments sont plus fréquentes que les accès.

LISTES CHAÎNÉES

- ▶ Taille arbitraire, limitée par la mémoire.
- ▶ Accès aux éléments de manière *séquentielle* via une suite de *pointeurs* : coûteux.
- ▶ Insertion et suppression en début de liste d'éléments en temps constant : simple.

LISTES CHAÎNÉES

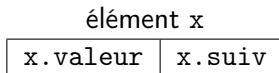
- ▶ Taille arbitraire, limitée par la mémoire.
- ▶ Accès aux éléments de manière *séquentielle* via une suite de *pointeurs* : coûteux.
- ▶ Insertion et suppression en début de liste d'éléments en temps constant : simple.

LISTES CHAÎNÉES

- ▶ Taille arbitraire, limitée par la mémoire.
- ▶ Accès aux éléments de manière *séquentielle* via une suite de *pointeurs* : coûteux.
- ▶ Insertion et suppression en début de liste d'éléments en temps constant : simple.

LISTES CHAÎNÉES

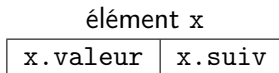
- **Principe** : les *éléments* de la liste sont des *objets* qui possèdent une *valeur* et un *pointeur* vers l'élément suivant dans la liste.



- Le dernier élément pointe vers None.
- La liste chaînée peut comporter un attribut *tete* qui désigne le premier élément de la liste.

LISTES CHAÎNÉES

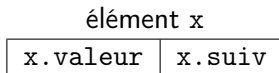
- **Principe** : les *éléments* de la liste sont des *objets* qui possèdent une *valeur* et un *pointeur* vers l'élément suivant dans la liste.



- Le dernier élément pointe vers None.
- La liste chaînée peut comporter un attribut *tete* qui désigne le premier élément de la liste.

LISTES CHAÎNÉES

- **Principe** : les *éléments* de la liste sont des *objets* qui possèdent une *valeur* et un *pointeur* vers l'élément suivant dans la liste.



- Le dernier élément pointe vers None.
- La liste chaînée peut comporter un attribut *tete* qui désigne le premier élément de la liste.

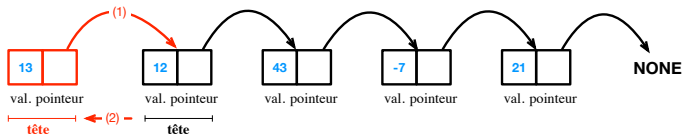
IMPLÉMENTATION : ÉLÉMENT DE LC (CELLULE)

```
1 class Cellule:
2     def __init__(self, valeur, suivant=None):
3         """creation d'une cellule
4
5         Args:
6             valeur (quelconque): valeur stockee
7             suiv (Cellule): cellule suivante dans la LC
8         """
9
10        self.valeur = valeur
11        self.suiv = suivant
```

IMPLÉMENTATION : LC

```
1 class LC:
2
3     def __init__(self):
4         """creation d'une liste chainee"""
5
6         self.tete = None # premier element de la liste
7
8     def est_vide(self):
9         """renvoie un booleen indiquant si
10        la liste est vide"""
11
12        return self.tete is None
```

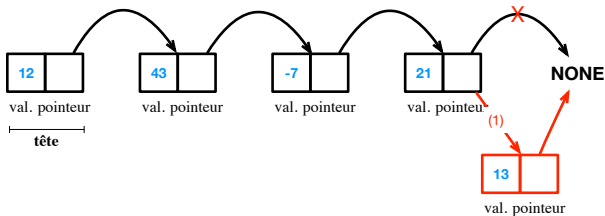
LISTES CHAÎNÉES : AJOUTER_DEBUT



LISTES CHAÎNÉES : AJOUTER_DEBUT

```
1  def ajouter_debut(self, x):  
2      """Ajoute x en tete de la liste"""  
3  
4      x.suiv = self.tete  
5      self.tete = x
```

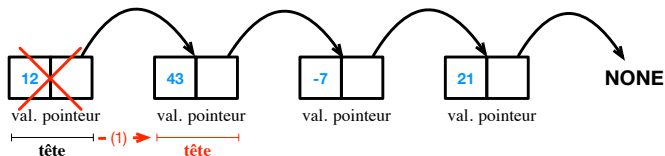
LISTES CHAÎNÉES : AJOUTER_FIN



LISTES CHAÎNÉES : AJOUTER_FIN

```
1  def ajouter_fin(self, x):  
2      """Ajoute x en queue de la liste"""  
3  
4      if self.est_vide():          # liste vide  
5          self.tete = x  
6          return  
7      m = self.tete  
8      while m.suiv is not None:  
9          m = m.suiv  
10     m.suiv = x                    # x.suiv = None par défaut
```

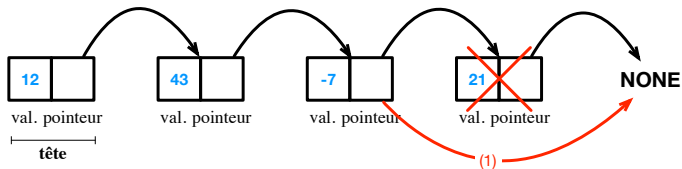
LISTES CHAÎNÉES : SUPPRIMER_DEBUT



LISTES CHAÎNÉES : SUPPRIMER_DEBUT

```
1  def supprimer_debut(self):  
2      """Supprime le 1er element de la liste  
3      et le renvoie"""  
4  
5      if self.est_vide():  
6          raise Exception("La liste est vide !")  
7      t = self.tete  
8      self.tete = t.suiv  
9      return t
```

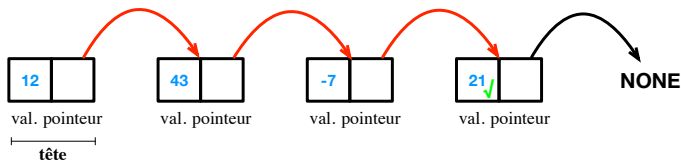
LISTES CHAÎNÉES : SUPPRIMER_FIN



LISTES CHAÎNÉES : SUPPRIMER_FIN

```
1  def supprimer_fin(self):
2      """Supprime le dernier element de la liste
3      et le renvoie"""
4
5      if self.est_vide():          # liste vide
6          raise Exception("La liste est vide !")
7      elif self.tete.suiv is None: # 1 seul element
8          e = self.tete
9          self.tete = None
10         return e
11     else:                         # au moins 2 elements
12         e = self.tete
13         e_courant = self.tete
14         while e.suiv is not None:
15             e_courant = e
16             e = e.suiv
17         e_courant.suiv = None
18         return e
```

LISTES CHAÎNÉES : RECHERCHER



LISTES CHAÎNÉES : RECHERCHER

```
1  def rechercher(self, valeur):
2      """Recherche valeur dans la liste et renvoie
3      l'element associe, ou None si absent"""
4
5      e = self.tete
6      while e is not None:
7          if e.valeur == valeur:
8              return e
9          e = e.suiv
10     return None
```

REMARQUES

- ▶ Les insertions et suppressions en début de liste sont simples : $O(1)$.
- ▶ Les insertions et suppressions en fin de liste ainsi que la recherche sont coûteuses : $O(n)$.
- ▶ Des piles et files *de tailles arbitraires* peuvent être implémentées par des listes chaînées (cf. section suivante).

REMARQUES

- ▶ Les insertions et suppressions en début de liste sont simples : $O(1)$.
- ▶ Les insertions et suppressions en fin de liste ainsi que la recherche sont coûteuses : $O(n)$.
- ▶ Des piles et files *de tailles arbitraires* peuvent être implémentées par des listes chaînées (cf. section suivante).

REMARQUES

- ▶ Les insertions et suppressions en début de liste sont simples : $O(1)$.
- ▶ Les insertions et suppressions en fin de liste ainsi que la recherche sont coûteuses : $O(n)$.
- ▶ Des piles et files *de tailles arbitraires* peuvent être implémentées par des listes chaînées (cf. section suivante).

COMPLEXITÉ DES OPÉRATIONS

Opérations	Tableaux	Listes chaînées
rechercher	$O(n)$	$O(n)$
ajouter/insérer	$O(n)$ ($O(1)^*$ en fin)	$O(1)/O(n)$ (début/fin)
supprimer	$O(n)$ ($O(1)^*$ en fin)	$O(1)/O(n)$ (début/fin)
min/max	$O(n)$	$O(n)$
accéder	$O(1)$	$O(n)$

* coût *amorti*, pour un tableau *dynamique* tel qu'une liste Python.

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.

- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.

- ▶ Une pile ou une file *S* comporte trois opérations (une requête et deux opérations pour être précis) :

est_vide(S) : teste si *S* est vide (requête) ;

ajouter(S,x) : ajoute l'élément *x* dans *S* (opération) ;

supprimer(S) : supprime l'élément de *S* (opération).

- ▶ **Remarque** : Pour ajouter, on spécifie l'élément *x* à ajouter. Pour supprimer, pas d'élément *x* spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.

- ▶ Une pile ou une file *S* comporte trois opérations (une requête et deux opérations pour être précis) :

est_vide(S) : teste si *S* est vide (requête) ;

ajouter(S,x) : ajoute l'élément *x* dans *S* (opération) ;

supprimer(S) : supprime l'élément de *S* (opération).

- ▶ **Remarque** : Pour *ajouter*, on spécifie l'élément *x* à ajouter. Pour *supprimer*, pas d'élément *x* spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.
- ▶ Une pile ou une file S comporte trois opérations (une requête et deux opérations pour être précis) :

`est_vide(S)` : teste si S est vide (requête) ;

`ajouter(S,x)` : ajoute l'élément x dans S (opération) ;

`supprimer(S)` : supprime l'élément de S (opération).

- ▶ **Remarque** : Pour `ajouter`, on spécifie l'élément x à ajouter. Pour `supprimer`, pas d'élément x spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.
- ▶ Une pile ou une file S comporte trois opérations (une requête et deux opérations pour être précis) :

`est_vide(S)` : teste si S est vide (requête) ;

`ajouter(S,x)` : ajoute l'élément x dans S (opération) ;

`supprimer(S)` : supprime l'élément de S (opération).

- ▶ Remarque : Pour ajouter, on spécifie l'élément x à ajouter. Pour supprimer, pas d'élément x spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.
- ▶ Une pile ou une file S comporte trois opérations (une requête et deux opérations pour être précis) :

`est_vide(S)` : teste si S est vide (requête) ;

`ajouter(S,x)` : ajoute l'élément x dans S (opération) ;

`supprimer(S)` : supprime l'élément de S (opération).

- ▶ **Remarque** : Pour `ajouter`, on spécifie l'élément x à ajouter. Pour `supprimer`, pas d'élément x spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.
- ▶ Une pile ou une file S comporte trois opérations (une requête et deux opérations pour être précis) :

`est_vide(S)` : teste si S est vide (requête) ;

`ajouter(S,x)` : ajoute l'élément x dans S (opération) ;

`supprimer(S)` : supprime l'élément de S (opération).

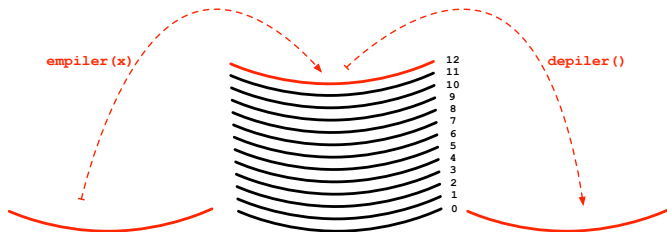
- ▶ Remarque : Pour ajouter, on spécifie l'élément x à ajouter. Pour supprimer, pas d'élément x spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

PILES ET FILES

- ▶ Une **pile (stack)** et une **file (queue)** sont des structures de données.
- ▶ Représentent des *ensembles dynamiques, ordonnés et de tailles arbitraires* d'éléments de même type.
- ▶ Une pile ou une file S comporte trois opérations (une requête et deux opérations pour être précis) :
 - `est_vide( $S$ )` : teste si S est vide (requête) ;
 - `ajouter( $S, x$ )` : ajoute l'élément x dans S (opération) ;
 - `supprimer( $S$ )` : supprime l'élément de S (opération).
- ▶ **Remarque** : Pour ajouter, on spécifie l'élément x à ajouter. Pour supprimer, pas d'élément x spécifié. Déterminé par la nature intrinsèque de l'ensemble : pile ou file (cf. suite).

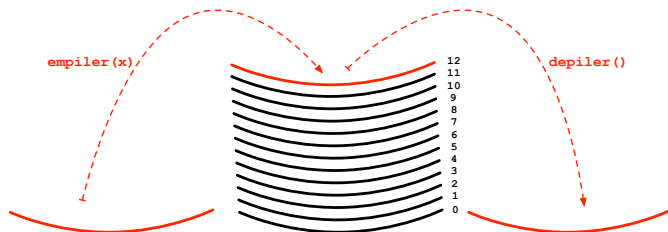
PILES

- ▶ Une **pile (stack)** implémente le principe :
dernier entré, premier sorti, Last In First Out, (LIFO) :
- ▶ Comme des assiettes sales que l'on empile : la dernière assiette empilée sera la première à être retirée pour être lavée.



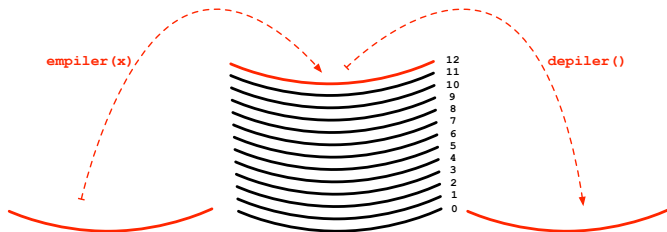
PILES

- ▶ Une **pile (stack)** implémente le principe :
dernier entré, premier sorti, Last In First Out, (LIFO) :
- ▶ Comme des assiettes sales que l'on empile : la dernière assiette empilée sera la première à être retirée pour être lavée.



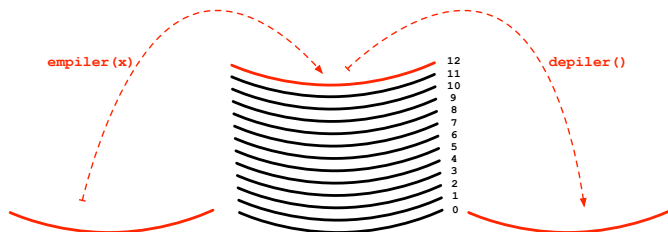
PILES

- ▶ Une **pile (stack)** implémente le principe :
dernier entré, premier sorti, Last In First Out, (LIFO) :
- ▶ Comme des assiettes sales que l'on empile : la dernière assiette empilée sera la première à être retirée pour être lavée.



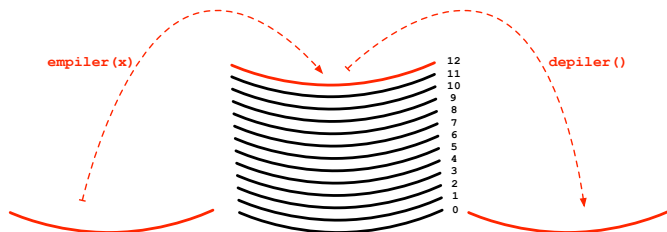
PILES : APPLICATIONS

- ▶ undo / redo dans des programmes ;
- ▶ page précédente / page suivante dans une navigation internet ;
- ▶ récursivité (pile d'appels de fonctions).



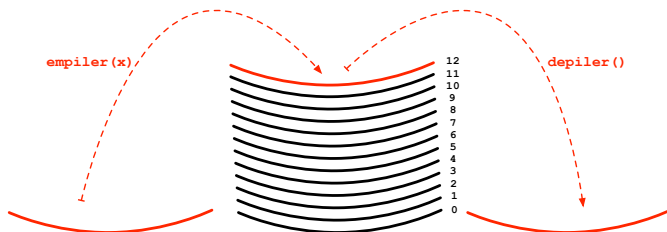
PILES : APPLICATIONS

- ▶ undo / redo dans des programmes ;
- ▶ page précédente / page suivante dans une navigation internet ;
- ▶ récursivité (pile d'appels de fonctions).



PILES : APPLICATIONS

- ▶ undo / redo dans des programmes ;
- ▶ page précédente / page suivante dans une navigation internet ;
- ▶ récursivité (pile d'appels de fonctions).



PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes `est_vide` et `est_pleine` :
 - Si `P.sommet == -1`, alors la pile est vide.
 - Si `P.sommet == n-1`, alors la pile est pleine.
- ▶ Opération **empiler** ou **push** :
 - on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
 - on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes `est_vide` et `est_pleine` :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :

on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.

- ▶ Opération **dépiler** ou **pop** :

on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :

Si `P.sommet == -1`, alors la pile est vide.

Si `P.sommet == n-1`, alors la pile est pleine.

- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :

on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

PILES

- ▶ Une **pile** d'au plus n éléments peut être implémentée par un tableau $P[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La pile possède un attribut `P.sommet` qui est l'indice du dernier élément inséré (indice et non élément).
- ▶ Requêtes **est_vide** et **est_pleine** :
Si `P.sommet == -1`, alors la pile est vide.
Si `P.sommet == n-1`, alors la pile est pleine.
- ▶ Opération **empiler** ou **push** :
on incrémente `P.sommet`, puis on assigne `P[P.sommet] = x`.
- ▶ Opération **dépiler** ou **pop** :
on décrémente `P.sommet` et on retourne `P[P.sommet+1]`.

IMPLÉMENTATION : PILE

```
1 class Pile:
2
3     def __init__(self, n):
4         """creation d'une pile d'au plus n elements
5
6         Args:
7             n (int): taille du tableau sous-jacent
8         """
9
10        self.n = n
11        self.P = [None] * n
12        self.sommet = -1           # -1 <=> pile vide
13
14    def est_vide(self):
15        return self.sommet == -1
16
17    def est_pleine(self):
18        return self.sommet == self.n - 1
```

PILES : EMPILER, DEPIILER

```
1  def empiler(self, x):
2      """Ajoute x au sommet de la pile"""
3
4      if self.est_pleine():
5          raise Exception("Débordement de la pile !")
6      self.sommet += 1
7      self.P[self.sommet] = x
8
9  def depiler(self):
10     """Supprime l'élément au sommet
11     de la pile et le renvoie"""
12
13     if self.est_vide():
14         raise Exception("La pile est vide !")
15     self.sommet -= 1
16     return self.P[self.sommet + 1]
```

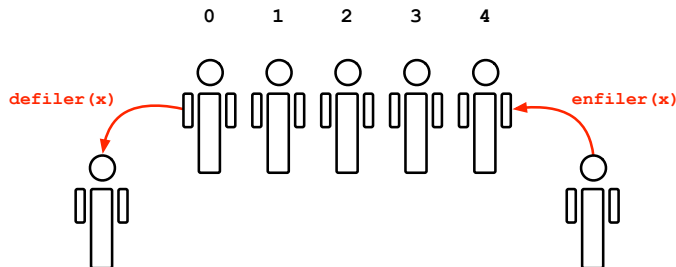
PILES : EXEMPLE D'EXÉCUTION ($n = 5$)

Opération	P	P.sommet
(initialisation)	$[-, -, -, -, -]$	-1
empiler(a)	$[a, -, -, -, -]$	0
empiler(b)	$[a, b, -, -, -]$	1
empiler(c)	$[a, b, c, -, -]$	2
depiler() $\rightarrow c$	$[a, b, c, -, -]$	1
empiler(d)	$[a, b, d, -, -]$	2

Remarque : depiler() n'efface rien physiquement : seul P.sommet change. La case contenant c est simplement écrasée au prochain empiler.

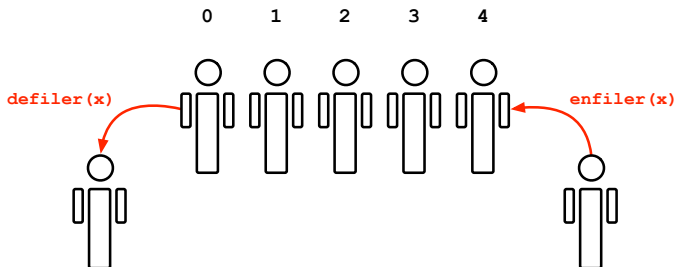
FILES

- ▶ Une **file** (**queue**) implémente le principe :
premier entré, premier sorti, First In First Out, (FIFO)
- ▶ Comme une file d'attente dans un magasin : la première personne arrivée sera la première servie.



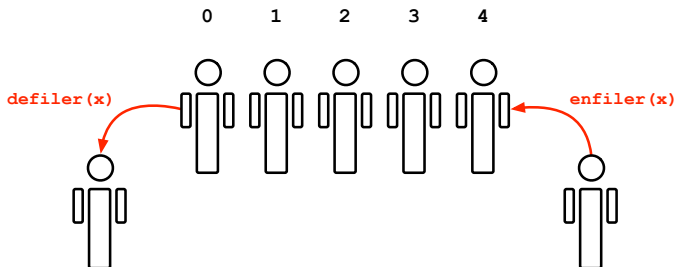
FILES

- ▶ Une **file** (**queue**) implémente le principe :
premier entré, premier sorti, First In First Out, (FIFO)
- ▶ Comme une file d'attente dans un magasin : la première personne arrivée sera la première servie.



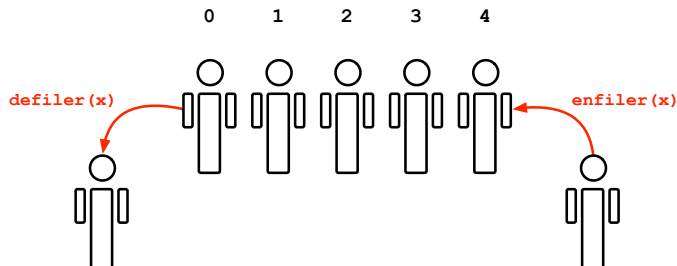
FILES

- ▶ Une **file** (**queue**) implémente le principe :
premier entré, premier sorti, First In First Out, (FIFO)
- ▶ Comme une file d'attente dans un magasin : la première personne arrivée sera la première servie.



FILES : APPLICATIONS

- Traiter des connexions, messages, processus, etc., dans leur ordre d'arrivée.



FILES

- ▶ Une **file** d'au plus $n-1$ éléments (et non n éléments) peut être implémentée par un tableau $F[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La file possède deux attributs :
 - $F.tete$: indice du prochain élément à sortir (élément courant)
 - $F.queue$: indice du prochain élément à entrer
- ▶ Les éléments de la file se trouvent aux emplacements $F.tete, F.tete + 1, \dots, F.queue - 1$ après quoi on boucle (l'emplacement 0 suit l'emplacement $n-1$).

FILES

- ▶ Une **file** d'au plus $n-1$ éléments (et non n éléments) peut être implémentée par un tableau $F[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La file possède deux attributs :
 - $F.tete$: indice du prochain élément à sortir (élément courant)
 - $F.queue$: indice du prochain élément à entrer
- ▶ Les éléments de la file se trouvent aux emplacements $F.tete, F.tete + 1, \dots, F.queue - 1$ après quoi on boucle (l'emplacement 0 suit l'emplacement $n-1$).

FILES

- ▶ Une **file** d'au plus $n-1$ éléments (et non n éléments) peut être implémentée par un tableau $F[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La file possède deux attributs :
 - $F.tete$: indice du prochain élément à sortir (élément courant)
 - $F.queue$: indice du prochain élément à entrer
- ▶ Les éléments de la file se trouvent aux emplacements $F.tete, F.tete + 1, \dots, F.queue - 1$ après quoi on boucle (l'emplacement 0 suit l'emplacement $n-1$).

FILES

- ▶ Une **file** d'au plus $n-1$ éléments (et non n éléments) peut être implémentée par un tableau $F[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La file possède deux attributs :
 - $F.tete$: indice du prochain élément à sortir (élément courant)
 - $F.queue$: indice du prochain élément à entrer
- ▶ Les éléments de la file se trouvent aux emplacements $F.tete, F.tete + 1, \dots, F.queue - 1$ après quoi on boucle (l'emplacement 0 suit l'emplacement $n-1$).

FILES

- ▶ Une **file** d'au plus $n-1$ éléments (et non n éléments) peut être implémentée par un tableau $F[0, \dots, n-1]$ ($0, \dots, n-1$ sont les indices).
- ▶ La file possède deux attributs :
 - $F.tete$: indice du prochain élément à sortir (élément courant)
 - $F.queue$: indice du prochain élément à entrer
- ▶ Les éléments de la file se trouvent aux emplacements $F.tete, F.tete + 1, \dots, F.queue - 1$ après quoi on boucle (l'emplacement 0 suit l'emplacement $n-1$).

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$
- ▶ Requêtes `est_vide` et `est_pleine` :
 - si $F.tete = F.queue$, alors la file est vide.
 - si $(F.queue + 1) \% n == F.tete$, alors la file est pleine (le $\%$ tient compte du bouclage).
- ▶ Opération `enfiler` ou `push` :
 - on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .
- ▶ Opération `défiler` ou `pop` :
 - on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$

- ▶ Requêtes **est_vide** et **est_pleine** :

si $F.tete = F.queue$, alors la file est vide.

si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).

- ▶ Opération **enfiler** ou **push** :

on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .

- ▶ Opération **défiler** ou **pop** :

on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$

- ▶ Requêtes **est_vide** et **est_pleine** :

si $F.tete = F.queue$, alors la file est vide.

si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).

- ▶ Opération **enfiler** ou **push** :

on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .

- ▶ Opération **défiler** ou **pop** :

on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$
- ▶ Requêtes **est_vide** et **est_pleine** :
si $F.tete = F.queue$, alors la file est vide.
si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).
- ▶ Opération **enfiler** ou **push** :
on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .
- ▶ Opération **défiler** ou **pop** :
on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$

- ▶ Requêtes **est_vide** et **est_pleine** :

si $F.tete = F.queue$, alors la file est vide.

si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).

- ▶ Opération **enfiler** ou **push** :

on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .

- ▶ Opération **défiler** ou **pop** :

on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$
- ▶ Requêtes **est_vide** et **est_pleine** :
si $F.tete = F.queue$, alors la file est vide.
si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).
- ▶ Opération **enfiler** ou **push** :
on assigne $F[F.queue] = x$ et on incrémente $F.queue \text{ modulo } n$.
- ▶ Opération **défiler** ou **pop** :
on retourne $F[F.tete]$ et on incrémente $F.tete \text{ modulo } n$.

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$
- ▶ Requêtes **est_vide** et **est_pleine** :
si $F.tete = F.queue$, alors la file est vide.
si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).
- ▶ Opération **enfiler** ou **push** :
on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .
- ▶ Opération **défiler** ou **pop** :
on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

FILES

- ▶ Au départ, on a $F.tete = F.queue = 0$
- ▶ Requêtes **est_vide** et **est_pleine** :
si $F.tete = F.queue$, alors la file est vide.
si $(F.queue + 1) \% n == F.tete$, alors la file est pleine
(le % tient compte du bouclage).
- ▶ Opération **enfiler** ou **push** :
on assigne $F[F.queue] = x$ et on incrémente $F.queue$ *modulo* n .
- ▶ Opération **défiler** ou **pop** :
on retourne $F[F.tete]$ et on incrémente $F.tete$ *modulo* n .

IMPLÉMENTATION : FILE

```
1 class File:
2
3     def __init__(self, n):
4         """creation d'une file d'au plus n-1 elements
5
6         Args:
7             n (int): taille du tableau sous-jacent
8         """
9
10        self.n = n
11        self.F = [None] * n
12        self.tete = 0      # prochain element a sortir
13        self.queue = 0     # prochain emplacement libre
14
15    def est_vide(self):
16        return self.tete == self.queue
17
18    def est_pleine(self):
19        return (self.queue + 1) % self.n == self.tete
```

FILES : ENFILER, DEFILER

```
1  def enfiler(self, x):
2      """Ajoute x en queue de la file"""
3
4      if self.est_pleine():
5          raise Exception("Débordement de la file !")
6      self.F[self.queue] = x
7      self.queue = (self.queue + 1) % self.n
8
9  def defiler(self):
10     """Supprime l'élément en tête
11     de la file et le renvoie"""
12
13     if self.est_vide():
14         raise Exception("La file est vide !")
15     x = self.F[self.tete]
16     self.tete = (self.tete + 1) % self.n
17     return x
```

FILES : EXEMPLE D'EXÉCUTION ($n = 5$)

Opération	F	F.tete	F.queue
(initialisation)	[-,-,-,-,-]	0	0
enfiler(a)	[a,-,-,-,-]	0	1
enfiler(b)	[a,b,-,-,-]	0	2
defiler() → a	[a,b,-,-,-]	1	2
enfiler(c)	[a,b,c,-,-]	1	3
enfiler(d)	[a,b,c,d,-]	1	4
enfiler(e)	[a,b,c,d,e]	1	0

À la dernière ligne, F.queue a **bouclé** de 4 à 0. La file contient alors b,c,d,e, soit $n-1 = 4$ éléments, et `est_pleine()` vaut True car $(0+1)\%5 == 1 == F.tete$: une case reste toujours inutilisée.